

Softwaredokumentation

LF8 – Daten systemübergreifend bereitstellen

Projekt: Lernmaterialverwaltung

Autor: Auric Ergeson Nitonde

Datum: 28 Mai 2026

Berufsschule Verden – Anwendungsentwickler

Inhaltsverzeichnis

1. Projektbeschreibung	3
2. Zielsetzung	3
3. Arbeitsprozess	3
Umsetzung	3
Reflexion	4
4. Funktionsübersicht	4
5. Implementierungsdetails	4
Architektur	4
Verwendete Bibliotheken	5
Speicherstrategie	5
Die sieben Datenbankabfragen	5
Quellcode-Kommentare	5
6. SQL-Code	6
7. ERD und ERM	8
Normalisierung	8
Entitäten und Beziehungen	9
8. Normalisierte Tabellenstruktur	10
9. Installation und Inbetriebnahme	12
Schritt 1 – Abhängigkeiten installieren	12
Schritt 2 – .env Datei anlegen	12
Schritt 3 – Datenbank einrichten	13
Schritt 4 – Anwendung starten	13

1. Projektbeschreibung

Die Lernmaterialverwaltung ist eine konsolenbasierte Python Anwendung, mit der Lehrkräfte und Auszubildende Lernmaterialien digital verwalten können. Über ein textbasiertes Menü lassen sich Dateien hochladen, suchen, herunterladen, kommentieren und löschen.

Alle Materialien werden in einer MySQL Datenbank gespeichert und mit Metadaten versehen: Titel, Dateityp, Themengebiet, Autor und Zeitstempel. Dateien unter 1 MB speichert das System direkt als Binärdaten in der Datenbank. Größere Dateien werden im Ordner uploads/ abgelegt und nur über den Pfad referenziert.

Das Projekt entstand im Rahmen von Lernfeld 8. Es ersetzt die bisherige Praxis, Materialien unstrukturiert in verschiedenen Verzeichnissen abzulegen oder per E-Mail weiterzugeben.

2. Zielsetzung

Ziel war es, eine einfache, datenbankgestützte Anwendung zu entwickeln, die Lernmaterialien zentral speicherbar und durchsuchbar macht. Statt Dateien in verschiedenen Ordnern zu verteilen, sollten alle Materialien an einem Ort liegen einheitlich mit Metadaten versehen und über ein Menü zugänglich.

Technisch waren MySQL als Datenbank, Python als Sprache und eine textbasierte Bedienung vorgegeben. Die Speicherstrategie kleine Dateien in der Datenbank, große Dateien im Dateisystem war Teil der Aufgabenstellung. Zusätzlich mussten mindestens sieben SQL Abfragen mit verschiedenen Join und Aggregationstypen umgesetzt werden.

3. Arbeitsprozess

Bevor eine einzige Zeile Python geschrieben wurde, haben wir die Datenbankstruktur geplant. Das war bewusst so entschieden: Wenn die Datenbank nicht stimmt, zieht sich das durch den ganzen Code.

Die größte Herausforderung war das Datenbankdesign selbst. Es war nicht sofort klar, wie viele Tabellen wir brauchen und wie sie zusammenhängen. Besonders die Frage, wie Tags gespeichert werden ob als Freitext in der material Tabelle oder in einer eigenen Tabelle hat einige Überlegungen erfordert. Wir haben uns für eine separate tag Tabelle mit Verbindungstabelle material_tag entschieden, weil das die sauberere Lösung im Sinne der Normalisierung ist.

Auch die Speicherstrategie musste vorab festgelegt werden: Dateien unter 1 MB kommen direkt in die Datenbank, größere Dateien ins Dateisystem. Diese Grenze ist im Code als Konstante (MAX_SIZE) definiert.

Umsetzung

Nach Fertigstellung des Datenbankschemas begann die Python Implementierung in drei Schritten: zuerst database.py (Verbindung und automatisches Tabellen Mapping), dann services.py (alle Datenbankfunktionen), zuletzt main.py (das Konsolenmenü).

Das Menü wurde so aufgebaut, dass jede Funktion eine eigene Funktion im Code hat. Das hat es einfach gemacht, einzelne Teile zu testen und später zu erweitern.

Reflexion

Was gut funktioniert hat: Das Konsolenmenü ist übersichtlich, alle sieben SQL Abfragen laufen stabil, und die Datenbankstruktur hat sich während der Entwicklung kaum verändert ein Zeichen dafür, dass die Planung am Anfang gründlich war.

Was wir beim nächsten Projekt anders machen würden: Früher mit dem Testen anfangen und nicht erst am Ende prüfen, ob alles funktioniert. Außerdem wäre eine einfache Fehlerprotokollierung hilfreich gewesen, um Probleme bei der Datenbankverbindung schneller zu erkennen.

Eine persönliche Anmerkung: Da Deutsch nicht meine Muttersprache ist, hat mein Ausbilder bei einzelnen Formulierungen in der Dokumentation geholfen und ich habe stellenweise Google Übersetzer genutzt. Der gesamte Quellcode sowie der überwiegende Teil der Dokumentation(91%) wurden eigenständig verfasst.

4. Funktionsübersicht

Die Anwendung bietet folgende Funktionen, erreichbar über das Hauptmenü:

Funktion	Beschreibung	Menü
Material hochladen	Eine Datei wird mit Titel, Thema und Autor versehen und gespeichert unter 1 MB direkt in der Datenbank, größere Dateien im Ordner uploads/.	[1]
Material herunterladen	Ein Material wird anhand der ID ausgewählt und in einen lokalen Ordner exportiert.	[2]
Materialien suchen	Die Liste lässt sich nach Titel, Themengebiet oder Autor filtern. Ergebnisse erscheinen als formatierte Tabelle.	[3]
Material löschen	Ein Material wird anhand der ID aus der Datenbank entfernt. Liegt die Datei im Dateisystem, wird sie dort ebenfalls gelöscht.	[4]
Kommentare verwalten	Kommentare zu einem Material können angezeigt, hinzugefügt oder gelöscht werden.	[5]
Listen & Übersichten	Materialien, Benutzer, Themen, Kategorien und Tags lassen sich vollständig als Tabelle einsehen.	[6]
Datenbank-Abfragen	Sieben SQL-Abfragen mit Joins und Aggregationen können einzeln oder alle auf einmal ausgeführt werden.	[7]
Neue Einträge erstellen	Benutzer, Themengebiete, Kategorien und Tags lassen sich direkt über das Menü anlegen.	[8]

5. Implementierungsdetails

Architektur

Das Programm ist in drei Schichten aufgeteilt. database.py stellt die Verbindung zur MySQL Datenbank her und lädt alle Tabellen über SQLAlchemy's automap_base() automatisch.

services.py enthält alle Datenbankfunktionen, von einfachen Abfragen bis zu den sieben Auswertungen. main.py ist die Benutzeroberfläche: ein Menü, das Eingaben entgegennimmt und die passenden Funktionen aus services.py aufruft.

Verwendete Bibliotheken

SQLAlchemy 2.0 wird als ORM eingesetzt. Statt SQL direkt zu schreiben, werden Python Objekte genutzt. Die automap Funktion liest bestehende Tabellen automatisch aus, ohne dass Modellklassen manuell definiert werden müssen. PyMySQL 1.1 ist der Treiber, der SQLAlchemy mit dem MySQL-Server verbindet.

rich 15.0 gibt formatierte Tabellen und farbigen Text in der Konsole aus, ohne grafische Oberfläche. Python dotenv 1.2 lädt Zugangsdaten aus einer .env Datei, damit Passwörter nicht direkt im Code stehen.

Speicherstrategie

Beim Upload wird die Dateigröße geprüft. Ist sie kleiner als 1.048.576 Bytes (1 MB), wird der Inhalt als LONGBLOB in der Tabelle material gespeichert und is_in_database auf TRUE gesetzt. Größere Dateien werden in den Ordner uploads/ kopiert, und nur der Pfad wird in der Spalte speicherort hinterlegt (is_in_database = FALSE). Beim Download entscheidet das System anhand dieses Flags, ob der Inhalt aus der Datenbank gelesen oder die Datei vom Dateisystem kopiert wird.

Die sieben Datenbankabfragen

Abfrage 1 => Materialien pro Themengebiet: COUNT-Aggregation mit GROUP BY. Zeigt, wie viele Materialien je Thema gespeichert sind (JOIN auf themengebiet).

Abfrage 2 => Durchschnittliche Dateigröße pro Dateityp: AVG-Aggregation mit GROUP BY auf material.dateityp.

Abfrage 3 => Materialien mit Themengebiet: INNER JOIN zwischen material und themengebiet.

Abfrage 4 => Materialien mit Kommentaren: INNER JOIN zwischen material und kommentar.

Abfrage 5 => Kommentaranzahl pro Material: INNER JOIN mit COUNT (JOIN + Aggregation kombiniert).

Abfrage 6 => Material mit Autor und Thema: Zwei JOINS auf benutzer und themengebiet.

Abfrage 7 => Vollständige Suche: Drei JOINS auf benutzer, themengebiet und kategorie, mit optionalen WHERE-Filtern nach Thema oder Autor.

Quellcode-Kommentare

Alle drei Python Dateien sind auf Deutsch kommentiert. Jede Funktion hat einen erklärenden Kommentar direkt über der Definition. Wichtige Konstanten und technische Entscheidungen sind ebenfalls dokumentiert. Einige Beispiele aus dem Quellcode:

```
# database.py
# Hier verbinden wir uns mit der MySQL-Datenbank.
# Die Zugangsdaten kommen aus der .env Datei.

# SessionLocal öffnet eine neue Verbindung wenn wir sie brauchen
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

```
# SQLAlchemy liest die Tabellen automatisch aus der Datenbank
Base = automap_base()

# services.py
# Dateien kleiner als 1 MB kommen direkt in die Datenbank
MAX_SIZE = 1 * 1024 * 1024

# Eine Datei hochladen - kleine Dateien in die DB, große auf die Festplatte
def upload_material(db, file_path, titel, beschreibung, ...):

# Abfrage 1: Wie viele Materialien gibt es pro Themengebiet? (COUNT)
def anzahl_material_pro_themen(db):

# main.py
# Menü zum Hochladen - Benutzer gibt Dateipfad und Infos ein
def upload_menu():

# Das Hauptmenü - hier startet alles
def hauptmenu():
```

6. SQL-Code

Der folgende SQL-Code erstellt die Datenbank lernmaterialverwaltung mit allen acht Tabellen und zwei Triggern:

```
DROP DATABASE IF EXISTS lernmaterialverwaltung;

CREATE DATABASE lernmaterialverwaltung
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE lernmaterialverwaltung;

CREATE TABLE themengebiet (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(100) NOT NULL UNIQUE,
    beschreibung TEXT
);

CREATE TABLE benutzer (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    username    VARCHAR(100) NOT NULL UNIQUE,
    email       VARCHAR(255) NOT NULL UNIQUE,
    rolle       ENUM('Lehrer', 'Azubi', 'Admin') NOT NULL DEFAULT 'Azubi',
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE kategorie (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(100) NOT NULL UNIQUE,
    beschreibung TEXT
);

CREATE TABLE tag (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(50) NOT NULL UNIQUE
);
```

```

CREATE TABLE material (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  titel       VARCHAR(255) NOT NULL,
  beschreibung TEXT,
  themengebiet_id INT NOT NULL,
  benutzer_id  INT NOT NULL,
  kategorie_id INT NOT NULL,
  erstellungsdatum TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  aenderungsdatum TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  dateiname    VARCHAR(255),
  dateityp     VARCHAR(50),
  dateigroesse BIGINT DEFAULT 0,
  speicherort  VARCHAR(500),
  inhalt       LONGBLOB,
  is_in_database BOOLEAN DEFAULT FALSE,

  FOREIGN KEY (themengebiet_id) REFERENCES themengebiet(id) ON DELETE RESTRICT,
  FOREIGN KEY (benutzer_id)     REFERENCES benutzer(id)      ON DELETE RESTRICT,
  FOREIGN KEY (kategorie_id)    REFERENCES kategorie(id)     ON DELETE RESTRICT
);

CREATE TABLE kommentar (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  material_id INT NOT NULL,
  autor_id    INT NOT NULL,
  text        TEXT NOT NULL,
  erstellungsdatum TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  aenderungsdatum TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  FOREIGN KEY (material_id) REFERENCES material(id) ON DELETE CASCADE,
  FOREIGN KEY (autor_id)   REFERENCES benutzer(id) ON DELETE RESTRICT
);

CREATE TABLE material_tag (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  material_id INT NOT NULL,
  tag_id      INT NOT NULL,

  FOREIGN KEY (material_id) REFERENCES material(id) ON DELETE CASCADE,
  FOREIGN KEY (tag_id)      REFERENCES tag(id)      ON DELETE CASCADE
);

CREATE TABLE version (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  material_id INT NOT NULL,
  autor_id    INT NOT NULL,
  version_nr  INT NOT NULL,
  aenderungsdatum TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  aenderungen TEXT,

  FOREIGN KEY (material_id) REFERENCES material(id) ON DELETE CASCADE,
  FOREIGN KEY (autor_id)   REFERENCES benutzer(id) ON DELETE RESTRICT
);

DELIMITER //

CREATE TRIGGER before_material_update
BEFORE UPDATE ON material
FOR EACH ROW
BEGIN
  SET NEW.aenderungsdatum = CURRENT_TIMESTAMP;

```

```

END //

CREATE TRIGGER before_kommentar_update
BEFORE UPDATE ON kommentar
FOR EACH ROW
BEGIN
    SET NEW.aenderungsdatum = CURRENT_TIMESTAMP;
END //

DELIMITER ;

```

7. ERD und ERM

Das folgende Entity Relationship Diagramm wurde mit DBeaver aus der bestehenden MySQL Datenbank generiert. Es zeigt alle acht Tabellen mit Spalten, Datentypen, Primärschlüsseln (fett markiert) und Fremdschlüsselbeziehungen (Linien mit Kardinalitätsangaben).

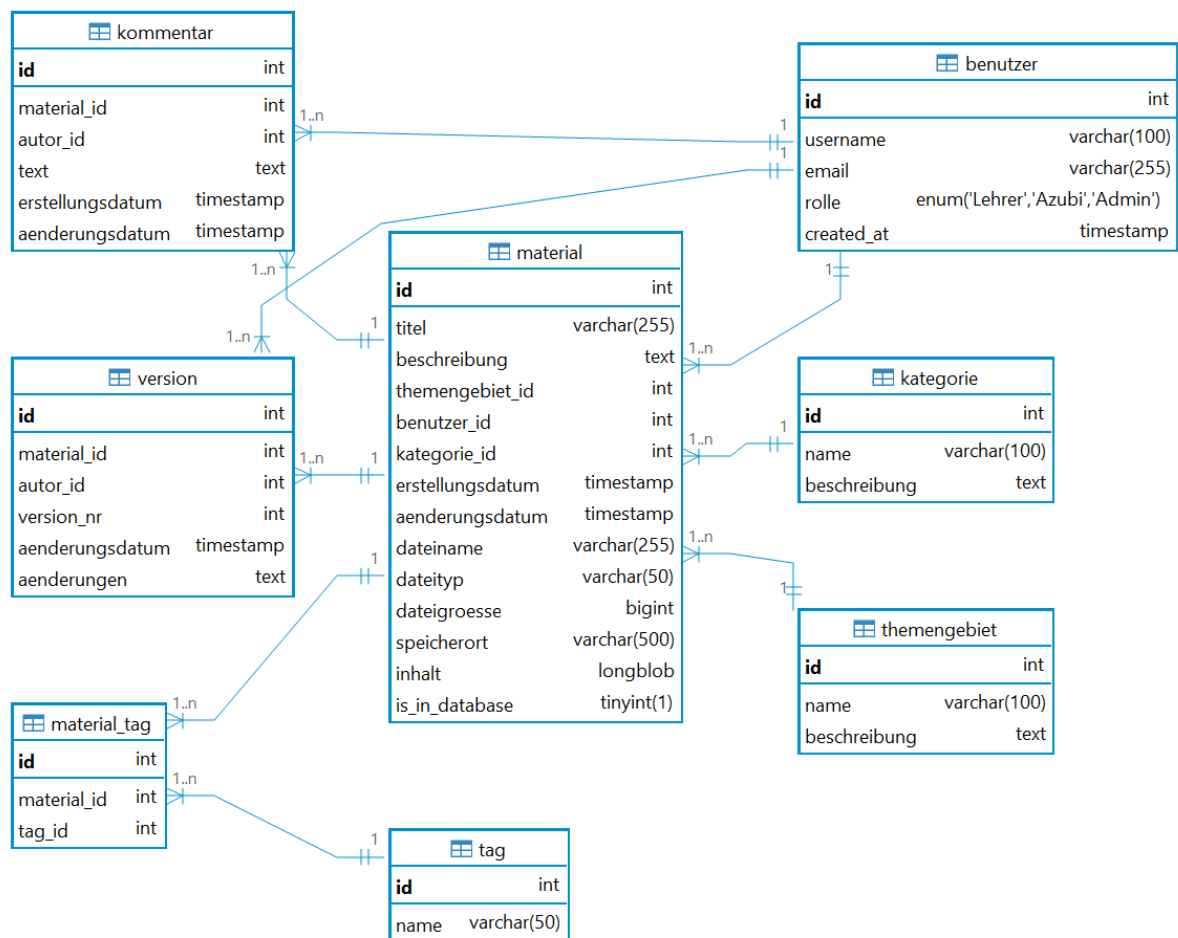


Abbildung 1: ERD der Datenbank lernmaterialverwaltung

Normalisierung

Die Datenbank wurde von Beginn an nach den Regeln der drei Normalformen entworfen. Ein erster Entwurf hätte Tags als Freitext Spalte in der material-Tabelle speichern können das wäre jedoch nicht normalisiert. Stattdessen wurden Tags in eine eigene Tabelle ausgelagert.

Erste Normalform (1NF): Alle Spalten enthalten atomare Werte ohne Wiederholungsgruppen. Tags sind nicht als kommaseparierte Liste in material gespeichert, sondern über die Verbindungstabelle material_tag referenziert.

Zweite Normalform (2NF): Da alle Tabellen einspaltige Primärschlüssel (id) verwenden, gibt es keine partiellen Abhängigkeiten. Jedes Nicht-Schlüssel-Attribut hängt vollständig vom Primärschlüssel ab.

Dritte Normalform (3NF): Es gibt keine transitiven Abhängigkeiten. Der Autor eines Materials wird als Fremdschlüssel (benutzer_id) referenziert, nicht als Freitext. Dasselbe gilt für themengebiet_id und kategorie_id – jede davon zeigt auf eine eigene Tabelle mit weiteren Attributen.

Entitäten und Beziehungen

benutzer – Benutzerinformationen

Attribute: id (PK, AI), username (UNIQUE), email (UNIQUE), rolle (Lehrer/Azubi/Admin), created_at

Beziehungen: 1:N zu material, 1:N zu kommentar, 1:N zu version.

themengebiet – Fachgebiete der Materialien

Attribute: id (PK, AI), name (UNIQUE), beschreibung

Beziehungen: 1:N zu material.

kategorie – Feinkategorisierung

Attribute: id (PK, AI), name (UNIQUE), beschreibung

Beziehungen: 1:N zu material.

tag – Freie Schlagwörter

Attribute: id (PK, AI), name (UNIQUE)

Beziehungen: N:M zu material über material_tag.

material – Zentrales Objekt der Anwendung

Attribute: id, titel, beschreibung, themengebiet_id (FK), benutzer_id (FK), kategorie_id (FK), erstellungsdatum, aenderungsdatum, dateiname, dateityp, dateigroesse, speicherort, inhalt (LONGBLOB), is_in_database

Beziehungen: Gehört zu 1 themengebiet, 1 benutzer, 1 kategorie. Hat 1:N zu kommentar und version. N:M zu tag über material_tag.

kommentar – Rückmeldungen zu Materialien

Attribute: id (PK, AI), material_id (FK), autor_id (FK), text, erstellungsdatum, aenderungsdatum

Beziehungen: Gehört zu 1 material und 1 benutzer.

material_tag – Verbindungstabelle (N:M)

Attribute: id (PK, AI), material_id (FK), tag_id (FK)

Beziehungen: Verbindet material und tag in einer N:M-Beziehung.

version – Versionshistorie

Attribute: id (PK, AI), material_id (FK), autor_id (FK), version_nr, aenderungsdatum, aenderungen

Beziehungen: Gehört zu 1 material und 1 benutzer.

8. Normalisierte Tabellenstruktur

Alle Tabellen liegen in der dritten Normalform (3NF) vor. Jede Spalte ist direkt vom Primärschlüssel abhängig, ohne partielle oder transitive Abhängigkeiten.

Tabelle: benutzer

Spalte	Datentyp	Schlüssel	Beschreibung
id	INT	PK, AI	Eindeutige Benutzer-ID
username	VARCHAR(100)	UNIQUE	Benutzername
email	VARCHAR(255)	UNIQUE	E-Mail-Adresse
rolle	ENUM		Rolle: Lehrer, Azubi oder Admin
created_at	TIMESTAMP		Erstellungsdatum (automatisch gesetzt)

Tabelle: themengebiet

Spalte	Datentyp	Schlüssel	Beschreibung
id	INT	PK, AI	Eindeutige Thema-ID
name	VARCHAR(100)	UNIQUE	Name des Themengebiets
beschreibung	TEXT		Optionale Beschreibung

Tabelle: kategorie

Spalte	Datentyp	Schlüssel	Beschreibung
id	INT	PK, AI	Eindeutige Kategorie-ID
name	VARCHAR(100)	UNIQUE	Name der Kategorie

Spalte	Datentyp	Schlüssel	Beschreibung
beschreibung	TEXT		Optionale Beschreibung

Tabelle: tag

Spalte	Datentyp	Schlüssel	Beschreibung
id	INT	PK, AI	Eindeutige Tag-ID
name	VARCHAR(50)	UNIQUE	Tag-Bezeichnung

Tabelle: material

Spalte	Datentyp	Schlüssel	Beschreibung
id	INT	PK, AI	Eindeutige Material-ID
titel	VARCHAR(255)		Titel des Lernmaterials
beschreibung	TEXT		Optionale Beschreibung
themengebiet_id	INT	FK	Verweis auf themengebiet(id)
benutzer_id	INT	FK	Verweis auf benutzer(id) Autor
kategorie_id	INT	FK	Verweis auf kategorie(id)
erstellungsdatum	TIMESTAMP		Zeitpunkt des Uploads (automatisch)
aenderungsdatum	TIMESTAMP		Letztes Änderungsdatum (automatisch)
dateiname	VARCHAR(255)		Originaldateiname
dateityp	VARCHAR(50)		MIME-Typ (z. B. application/pdf)
dateigroesse	BIGINT		Dateigröße in Bytes
speicherort	VARCHAR(500)		Dateipfad (bei Dateisystemspeicherung)
inhalt	LONGBLOB		Dateiinhalt (bei Dateien unter 1 MB)
is_in_database	BOOLEAN		TRUE = Inhalt in DB, FALSE = Dateisystem

Tabelle: kommentar

Spalte	Datentyp	Schlüssel	Beschreibung
id	INT	PK, AI	Eindeutige Kommentar-ID
material_id	INT	FK	Verweis auf material(id)
autor_id	INT	FK	Verweis auf benutzer(id)
text	TEXT		Kommentartext
erstellungsdatum	TIMESTAMP		Erstellungszeitpunkt

Spalte	Datentyp	Schlüssel	Beschreibung
aenderungsdatum	TIMESTAMP		Letztes Änderungsdatum

Tabelle: material_tag

Spalte	Datentyp	Schlüssel	Beschreibung
id	INT	PK, AI	Eindeutige Verbindungs-ID
material_id	INT	FK	Verweis auf material(id)
tag_id	INT	FK	Verweis auf tag(id)

Tabelle: version

Spalte	Datentyp	Schlüssel	Beschreibung
id	INT	PK, AI	Eindeutige Versions-ID
material_id	INT	FK	Verweis auf material(id)
autor_id	INT	FK	Verweis auf benutzer(id)
version_nr	INT		Versionsnummer
aenderungsdatum	TIMESTAMP		Zeitpunkt der Änderung
aenderungen	TEXT		Beschreibung der vorgenommenen Änderungen

9. Installation und Inbetriebnahme

Um die Anwendung lokal auszuführen, werden Python 3.10 (oder neuer), MySQL 8.0 und pip benötigt.

Schritt 1 – Abhängigkeiten installieren

Im Projektverzeichnis folgenden Befehl ausführen:

```
pip install -r requirements.txt
```

Schritt 2 – .env Datei anlegen

Im Projektverzeichnis eine Datei mit dem Namen .env erstellen und folgende Werte eintragen:

```
DB_DRIVER=mysql+pymysql
DB_USERNAME=root
DB_PASSWORD=IhrPasswort
DB_HOST=localhost
DB_PORT=3306
DB_NAME=lernmaterialverwaltung
```

Schritt 3 – Datenbank einrichten

Das SQL-Schema in MySQL importieren (einmalig, erstellt die Datenbank und alle Tabellen):

```
mysql -u root -p < sql/schema.sql
```

Schritt 4 – Anwendung starten

In den app-Ordner wechseln und das Programm starten:

```
cd app  
python main.py
```

Das Hauptmenü erscheint direkt im Terminal. Die Anwendung läuft vollständig textbasiert und benötigt keine grafische Oberfläche.